

Contents

1	PHP Functions	1
1.1	Defining and Calling Functions	1
1.2	Function Arguments, Value and Reference, and Return Values	2
1.2.1	Passing Arguments	2
1.2.2	Value vs. Reference	2
1.2.3	Return Values	3
1.3	Built-in PHP Functions	3
1.3.1	String Functions	3
1.3.2	Array Functions	3
1.3.3	Date and Time Functions	3
1.4	Summary	4
2	Anex I: List of predefined and standard functions	4
2.1	String Functions	4
2.2	Array Functions	4
2.3	Mathematical Functions	5
2.4	Date and Time Functions	5
2.5	File Handling Functions	5
2.6	Miscellaneous Functions	5

1 PHP Functions

Functions are one of the most essential constructs in programming. They encapsulate code for specific tasks, enabling code reuse, logical organization, and the simplification of complex operations. In this chapter, we'll delve into defining and calling functions in PHP, managing data passed to and from these functions, and using some of the many built-in PHP functions.

1.1 Defining and Calling Functions

A function is a designated block of code designed to perform a particular task. In PHP, a function is defined using the `function` keyword, followed by a unique function name and a set of parentheses. Optional parameters can be specified within these parentheses, and these act as input variables for the function. The function's body, enclosed within curly braces, contains the code executed upon the function's invocation.

Here's an example of a simple function definition that outputs a greeting message:

```
<?=  
function sayHello() {  
    echo "Hello, world!";  
}
```

To call this function, we use its name followed by parentheses. If the function has parameters, the corresponding arguments are passed within these parentheses. An argument is a value supplied to the function upon its call.

Here's how we call the `sayHello` function:

```
<?=  
    sayHello(); // Outputs: Hello, world!
```

1.2 Function Arguments, Value and Reference, and Return Values

1.2.1 Passing Arguments

Parameters allow functions to accept inputs. These inputs are termed as arguments when a function is called. Each argument corresponds to a function's parameter by order.

Consider a function that greets a user by name:

```
<?=  
    function greet($name) {  
        echo "Hello, $name!";  
    }  
  
    greet("Alice"); // Outputs: Hello, Alice!
```

In this example, "Alice" is the argument passed to the `greet` function.

1.2.2 Value vs. Reference

In PHP, arguments can be passed by value or by reference. Passing by value means the function works with a copy of the argument, whereas passing by reference means the function works directly with the original variable.

Example of passing by value:

```
<?=  
    function addOne($number) {  
        $number++;  
        echo $number;  
    }  
  
    $originalNumber = 5;  
    addOne($originalNumber); // Outputs: 6  
    echo $originalNumber; // Still outputs: 5
```

In contrast, passing by reference:

```
<?=  
    function addOneByRef(&$number) {  
        $number++;  
    }  
  
    $originalNumber = 5;  
    addOneByRef($originalNumber);  
    echo $originalNumber; // Outputs: 6
```

The `&` symbol in the parameter list indicates that the argument is passed by reference.

1.2.3 Return Values

Functions can return values using the `return` statement. This ends the function's execution and optionally passes back a value.

For example:

```
<?=  
function sum($a, $b) {  
    return $a + $b;  
}  
  
$total = sum(5, 3);  
echo $total; // Outputs: 8
```

1.3 Built-in PHP Functions

PHP offers an extensive collection of built-in functions that perform a wide range of tasks.

1.3.1 String Functions

String manipulation is a common requirement, and PHP provides many functions to handle strings. Functions like `strlen`, `str_replace`, and `substr` are frequently used.

Example of string functions:

```
<?=  
$text = "Hello, world!";  
echo strlen($text); // Outputs the length of the string  
echo str_replace("world", "PHP", $text); // Replaces 'world' with 'PHP'
```

1.3.2 Array Functions

Arrays are essential in PHP, and functions like `array_push`, `sort`, and `count` are integral for array manipulation.

Example of array functions:

```
<?=  
$fruits = ["apple", "banana"];  
array_push($fruits, "orange"); // Adds 'orange' to the array  
sort($fruits); // Sorts the array  
echo count($fruits); // Outputs the number of elements in the array
```

1.3.3 Date and Time Functions

Handling dates and times is another common requirement. Functions such as `date` and `strtotime` are useful for formatting and manipulating date and time.

Example of date and time functions:

```
<?=  
echo date("Y-m-d"); // Outputs the current date echo  
strtotime("next Monday"); // Converts the time string to a Unix  
timestamp
```

1.4 Summary

Functions in PHP are powerful tools for organizing and reusing code. Understanding how to define, call, and manipulate data with functions is key to effective PHP programming. Additionally, leveraging PHP's built-in functions can greatly simplify many common tasks in web development.

In the next sections, we will explore further into more advanced aspects of PHP functions, including anonymous functions, variable scope, and recursive functions. Stay tuned to deepen your understanding and enhance your PHP programming skills.

- Anex I Below is a list of commonly used PHP built-in functions, categorized by their functionality. This list is by no means exhaustive, as PHP offers a vast library of built-in functions, but it covers some of the most frequently used ones in various aspects of PHP programming:

2 Anex I: List of predefined and standard functions

2.1 String Functions

1. `strlen($string)` - Returns the length of a string.
2. `str_replace($search, $replace, $subject)` - Replaces all occurrences of a search string with a replacement string.
3. `strpos($haystack, $needle)` - Finds the position of the first occurrence of a substring in a string.
4. `strtolower($string)` - Converts a string to lowercase.
5. `strtoupper($string)` - Converts a string to uppercase.
6. `substr($string, $start, $length)` - Returns a part of a string.
7. `trim($string)` - Strips whitespace from the beginning and end of a string.

2.2 Array Functions

1. `array_push(&$array, $value1, $value2, ...)` - Pushes one or more elements onto the end of an array.
2. `sort(&$array)` - Sorts an array.
3. `count($array)` - Counts all elements in an array, or something in an object.
4. `array_merge($array1, $array2, ...)` - Merges one or more arrays.
5. `array_slice($array, $offset, $length)` - Extracts a slice of the array.
6. `in_array($needle, $haystack)` - Checks if a value exists in an array.

7. `array_keys($array)` - Returns all the keys or a subset of the keys of an array.

2.3 Mathematical Functions

1. `abs($number)` - Returns the absolute value of a number.
2. `ceil($value)` - Rounds a number up to the nearest integer.
3. `floor($value)` - Rounds a number down to the nearest integer.
4. `rand($min, $max)` - Generates a random integer.
5. `max($value1, $value2, ...)` - Finds the highest value.
6. `min($value1, $value2, ...)` - Finds the lowest value.
7. `round($val, $precision)` - Rounds a float.

2.4 Date and Time Functions

1. `date($format, $timestamp)` - Formats a local time/date.
2. `strtotime($time)` - Parses an English textual datetime into a Unix timestamp.
3. `time()` - Returns the current time as a Unix timestamp.
4. `mktime($hour, $minute, $second, $month, $day, $year)` - Gets Unix timestamp for a date.
5. `date_default_timezone_set($timezone_identifier)` - Sets the default timezone used by all date/time functions.

2.5 File Handling Functions

1. `fopen($filename, $mode)` - Opens file or URL.
2. `fclose($handle)` - Closes an open file pointer.
3. `fwrite($handle, $string)` - Writes to a file.
4. `file_get_contents($filename)` - Reads entire file into a string.
5. `file_put_contents($filename, $data)` - Write data to a file.
6. `file_exists($filename)` - Checks whether a file or directory exists.

2.6 Miscellaneous Functions

1. `isset($var)` - Determine if a variable is set and is not NULL.
2. `empty($var)` - Determine whether a variable is empty.
3. `die($message)` - Equivalent to `exit()`, outputs a message and terminates the current script.
4. `var_dump($variable)` - Dumps information about a variable.

5. `gettype($var)` - Gets the type of a variable.

This list serves as a starting point for exploring PHP's built-in functions. Each function has its own set of parameters and behaviors, which can be further explored in the PHP official documentation for more detailed information and usage examples.